

Analisi di Dati sui Voli Aerei con Apache Spark

una Pipeline di Batch Processing su Architettura Containerizzata

Sistemi e Architetture per Big Data A.A. 2025/26

Valentina Jin
Ingegneria Informatica
Università di Roma Tor Vergata
valentina.jin@students.uniroma2.eu

Elisa Marzioli
Ingegneria Informatica
Università di Roma Tor Vergata
elisa.marzioli@students.uniroma2.eu

Abstract—Questo progetto presenta una pipeline di batch processing per l'analisi di circa 2,2 milioni di record sui voli aerei statunitensi (gennaio-aprile 2025), forniti dal Dipartimento dei Trasporti degli Stati Uniti (BTS). L'architettura containerizzata tramite Docker Compose orchestra i componenti Apache NiFi, HDFS, Apache Spark (PySpark), Redis e Grafana, con Apache Airflow come strumento di orchestrazione. Vengono implementate tre query analitiche sui ritardi e le cancellazioni dei voli. L'obiettivo dello studio è determinare l'impatto di determinati parametri di configurazione sulle performance della pipeline di batch processing.

Parole chiave—Spark, PySpark, HDFS, Docker, NiFi, Redis, Grafana, Airflow, batch processing, big data.

I. INTRODUCTION

L'analisi di grandi volumi di dati nel settore dei trasporti rappresenta un caso d'uso emblematico per le tecnologie di Big Data processing. In questo contesto, il progetto si propone di costruire una pipeline completa che, a partire da dati grezzi sui voli, consenta di estrarre informazioni significative su ritardi, cancellazioni e loro cause attraverso tre query analitiche implementate con Apache Spark [1].

Il dataset, fornito dal Dipartimento dei Trasporti degli Stati Uniti (BTS) [2], copre il periodo gennaio-aprile 2025 e include informazioni su compagnie aeree, orari, ritardi in partenza e in arrivo, stato del volo (cancellato/deviato) e cause di ritardo. La scelta delle tecnologie riflette un approccio orientato alla scalabilità e alla riproducibilità: Docker Compose [3] per la containerizzazione, Apache Airflow [4] per l'orchestrazione, Apache NiFi [5] come il tool per la data ingestion, Parquet come formato di storage intermedio su HDFS [6], Redis [7] come il database di salvaggio dei risultati analitici, e Grafana [8] per la visualizzazione dei risultati.

II. ARCHITETTURA DEL SISTEMA

L'architettura è completamente containerizzata tramite Docker Compose e si articola in sei componenti principali, orchestrati da Apache Airflow tramite il DAG `sabd_project1_pipeline`.

La Fig. 1 illustra il flusso dati e le interazioni tra i componenti.

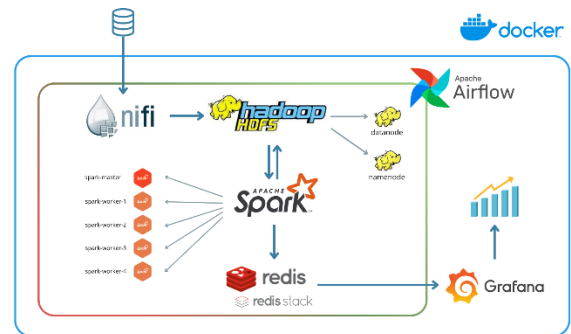


Figura 1 Schema dell'architettura del sistema distribuito

I componenti principali sono:

- **Apache NiFi:** si occupa della data ingestion automatizzata. Scarica i dataset mensili dall'URL in formato .tar.gz, li decompone e li carica su HDFS nel percorso `/data/raw/flights/csv/`, fungendo da punto di ingresso controllato per i dati grezzi nella pipeline.
- **Hadoop Distributed File System (HDFS):** funge da storage distribuito persistente dei dati grezzi, processati e convertiti in formato Parquet, nonché dell'output. Nella configurazione attuale è costituito da un datanode e un namenode, dato il deploy locale.
- **Apache Spark:** costituisce il nucleo computazionale del sistema. Esegue il preprocessing e le tre query analitiche tramite PySpark, sfruttando un cluster in modalità Standalone con un master e numero variabile di worker, da 1 a 4. Le risorse per ogni Spark Worker vengono limitate a 1GB di RAM e 1 core, per permettere un'analisi di scalabilità anche in condizioni di deploy locale.
- **Redis:** funge da serving layer tra Spark e Grafana. I risultati aggregati prodotti dalle query vengono esportati in Redis Stack [9] per rendere i dati direttamente accessibili ai pannelli Grafana.
- **Grafana:** piattaforma open-source per la visualizzazione interattiva di dati. Accede a Redis tramite il plugin Redis Data Source e presenta una dashboard dedicata per ciascuna query, con pannelli time-series, bar chart e tabelle.

- **Apache Airflow:** orchestratore della pipeline end-to-end. Il DAG `sabd_project1_pipeline` definisce le dipendenze tra i task, garantendo la corretta sequenzializzazione (ingestion → preprocessing → query → export) e il monitoraggio dell'esecuzione.
- **Docker Compose:** definisce e orchestra l'intero ambiente multi-container del sistema. Tramite un unico file `docker-compose.yml`, specifica i servizi (NiFi, HDFS namenode e datanode, Spark master e worker, Redis Stack, Grafana, Airflow), le dipendenze tra di essi, la rete interna (`sabd_net`) e i volumi persistenti, garantendo riproducibilità e portabilità dell'infrastruttura.

III. DATA PIPELINE

A. Data Ingestion (Apache NiFi)

Apache NiFi gestisce il processo di acquisizione dei dati tramite il Process Group `sabd-ingest-flights-to-hdfs` (Figura 2). Il flusso NiFi automatizza il download dei dataset mensili in formato `.tar.gz`, la decompressione e il caricamento dei CSV su HDFS nel percorso `hdfs://namenode:9000/data/raw/flights/csv/`.

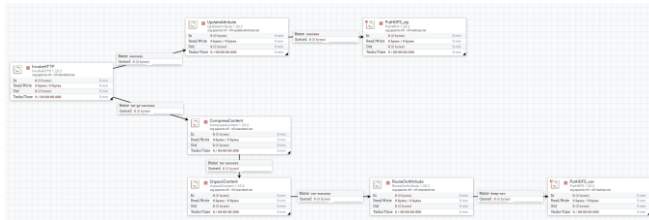


Figura 2 Dataflow NiFi - Data ingestion

La scelta di NiFi rispetto ad alternative come Apache Kafka o Pulsar è motivata dalla natura batch del processo (4 file mensili, non un flusso continuo): NiFi offre un'interfaccia grafica per la configurazione del flusso, gestione nativa degli errori e retry, e processor specializzati per operazioni su file e HDFS, senza richiedere la complessità di un sistema di messaggistica distribuita.

B. Preprocessing

Lo script `preprocess.py` legge i CSV raw da HDFS applicando uno schema esplicito e produce un dataset Parquet processato salvato in `hdfs://namenode:9000/data/processed/flights/parquet`. Le operazioni principali includono:

- Selezione delle 14 colonne utili alle tre query (riduzione del volume trasportato).
- Cast esplicito dei tipi: `CANCELLED` e `DIVERTED` convertiti da `DoubleType` a `IntegerType`.
- Eliminazione di righe con `OP_UNIQUE_CARRIER` nullo (dati invalidi senza compagnia identificabile).
- Imputazione a 0 dei valori null nelle colonne delle cause di ritardo (`CARRIER_DELAY`, `WEATHER_DELAY`, `NAS_DELAY`, `SECURITY_DELAY`, `LATE_AIRCRAFT_DELAY`): il BTS registra queste cause solo quando `ARR_DELAY > 15` minuti; trattare il valore mancante come 0 mantiene la consistenza del denominatore nel calcolo delle medie.

Il formato Parquet è stato scelto rispetto a CSV, ORC e Avro per il suo supporto nativo alla column pruning (solo le colonne richieste vengono lette da disco), la compressione efficiente e la compatibilità con Spark.

Si è scelto di non effettuare un partizionamento (ad esempio su base mensile) del dataset in formato Parquet, in quanto l'utilizzo di HDFS, nativamente ottimizzato per file di grandi dimensioni, avrebbe rappresentato una minaccia alle performance della pipeline.

Il dataset risultante è un unico Parquet logico gestito da Spark tramite part-file.

C. Query Execution (Q1, Q2, Q3)

Le tre query analitiche sono state implementate utilizzando le API `DataFrame` di PySpark come implementazione principale. Il `DataFrame` API sfrutta l'ottimizzatore Catalyst di Spark, che genera piani di esecuzione fisici ottimizzati applicando automaticamente tecniche come column pruning, predicate pushdown e ottimizzazione degli shuffle.

In una seconda fase, ciascuna query è stata reimplementata utilizzando le API RDD di basso livello, che operano senza ottimizzazioni automatiche ma offrono un controllo esplicito sul flusso di dati e sulla serializzazione. Il confronto sistematico tra le due implementazioni, condotto tramite un benchmark dedicato, avviene nella sezione V.

Q1 - Statistiche Mensili per Compagnia

Q1 analizza i voli delle compagnie American Airlines (AA) e Delta Air Lines (DL) su base mensile, calcolando per i voli non cancellati: media, minimo e massimo di `DEP_DELAY` e `ARR_DELAY`; e per tutti i voli del gruppo: cancellation rate (percentuale di voli cancellati sul totale).

Q1 - DataFrame:

1. Lettura del file Parquet su HDFS (`spark.read.parquet`);
2. Proiezione delle colonne necessarie (`select`);
3. Filtro sui carrier target AA e DL (`filter + isin`);
4. Aggregazione per coppia (mese, compagnia) (`groupBy` su `MONTH`, `OP_UNIQUE_CARRIER`):
 - conteggio totale voli, voli cancellati e non cancellati (`F.count`, `F.sum`);
 - valore medio, minimo e massimo di `DEP_DELAY` e `ARR_DELAY` sui soli voli non cancellati (`F.avg`, `F.min`, `F.max` con `F.when`);
5. Calcolo della cancellation rate come colonna derivata (`withColumn`);
6. Arrotondamento dei risultati e ordinamento per compagnia e mese (`select`, `round`, `orderBy`);
7. Salvataggio in formato CSV su HDFS e in locale (`save_csv`).

L'esecuzione delle trasformazioni è forzata tramite le azioni `count` e `collect` (`save_csv`).

Q1 - RDD:

La logica di query è identica alla versione `DataFrame`. Le differenze principali riguardano:

- Conversione in RDD tramite `.rdd` dopo la lettura del Parquet, con cache del risultato filtrato sui carrier target;
- Tre rami + join: le statistiche di departure delay, arrival delay (per le quali consideriamo solo voli non cancellati) e di cancellation rate (calcolate invece su tutti i voli) vengono calcolate separatamente su tre rami distinti, poi ricongiunti via `join` sulla chiave (`month`, `carrier`);
- Aggregazione tramite `combineByKey` (per `mean/min/max` di `DEP_DELAY` e `ARR_DELAY`) e `reduceByKey` (per i conteggi di `cancellation rate`). Si è preferita la `combineByKey` alla `groupByKey` perché essa aggrega parzialmente all'interno di ogni partizione prima dello shuffle, riducendo la quantità di dati trasferiti tra i nodi.

Q2 - Top-10 Compagnie per Ritardo in Arrivo

Q2 analizza tutte le compagnie aeree (voli non cancellati e non devianti con `ARR_DELAY` non nullo) calcolando: `ARR_DELAY` medio e medie delle 5 componenti di ritardo (`CARRIER_DELAY`, `WEATHER_DELAY`, `NAS_DELAY`, `SECURITY_DELAY`, `LATE_AIRCRAFT_DELAY`). Una seconda fase filtra le compagnie con almeno 500 voli validi e seleziona le top-10 per `ARR_DELAY` medio decrescente. Il DataFrame intermedio di tutte le compagnie viene inserito in cache, poiché viene riutilizzato per la costruzione della top-10.

Q2 - DataFrame:

1. Lettura del file Parquet su HDFS (`spark.read.parquet`);
2. Proiezione delle colonne necessarie (`select`);
3. Filtraggio dei voli validi: non cancellati, non devianti, `ARR_DELAY` non nullo (`filter`);
4. Aggregazione per compagnia aerea (`groupBy` su `OP_UNIQUE_CARRIER`):
5. numero di voli validi (`F.count`);
6. `ARR_DELAY` medio e medie delle 5 componenti di ritardo (`F.avg`);
7. Arrotondamento dei risultati e ordinamento alfabetico (`select, round, orderBy`);
8. Materializzazione in cache del DataFrame intermedio (`cache, count`);
9. Filtraggio compagnie con almeno 500 voli validi (`filter`);
10. Ordinamento decrescente per `ARR_DELAY` medio e selezione top-10 (`orderBy, limit`);
11. Salvataggio dei due output in formato CSV su HDFS e in locale (`save_csv`).

Q2 - RDD:

La logica di query è identica alla versione DataFrame. Le differenze principali riguardano:

- Conversione in RDD tramite `.rdd` dopo la lettura del Parquet;

- Aggregazione tramite `combineByKey` (pattern `create_combiner / merge_value / merge_combiners`);
- Selezione top-10 tramite `zipWithIndex + filter`.
- Liberazione dei dati in cache tramite `unpersist`.

Q3 - Percentili di Ritardo per Fascia Oraria

Q3 analizza i voli non cancellati con `DEP_DELAY` non nullo delle compagnie AA, DL, UA (United Airlines) e WN (Southwest Airlines), producendo due output: (1) percentili P25, P50, P75, P90 di `DEP_DELAY` per coppia (compagnia, fascia oraria 0-23); (2) min e max assoluti di `DEP_DELAY` per compagnia. Solo voli non cancellati con `DEP_DELAY` non nullo e ora valida (0-23) vengono inclusi.

Q3 - DataFrame:

1. Lettura del file Parquet su HDFS (`spark.read.parquet`);
2. Proiezione delle colonne necessarie (`select`);
3. Filtro sui carrier target, voli non cancellati e `DEP_DELAY` non nullo (`filter`); derivazione della colonna `hour` da `CRS_DEP_TIME` via divisione intera per 100 (`withColumn, cast`); filtro su ore valide 0-23;
4. Materializzazione in cache del DataFrame filtrato (`cache`);
5. Output 1: aggregazione per (compagnia, ora) (`groupBy`): conteggio voli e percentili P25/P50/P75/P90 tramite la funzione nativa di Spark `F.percentile_approx` (algoritmo Greenwald-Khanna, `accuracy=1000`, errore relativo $\leq 0,1\%$); ordinamento per compagnia e ora (`orderBy`);
6. Output 2: aggregazione per compagnia (`groupBy`): min e max assoluti di `DEP_DELAY` (`F.min, F.max`); ordinamento per compagnia (`orderBy`);
7. Salvataggio dei due output in formato CSV su HDFS e in locale (`save_csv`).

Q3 - RDD:

La logica di query è identica alla versione DataFrame. Le differenze principali riguardano:

- Conversione in RDD tramite `.rdd` dopo la lettura del Parquet;
- Percentili: ogni partizione costruisce un TDigest locale tramite `t-digest [10]` via `mapPartitions`; i digest parziali vengono fusi tra partizioni con `reduceByKey + update_centroids_from_list`; i percentili P25/P50/P75/P90 vengono estratti sul driver tramite `td.percentile()`;
- Min/Max: aggregazione tramite `combineByKey`;

Questo approccio evita il collect dei dati raw ma introduce overhead di comunicazione driver-executor.

La scelta degli algoritmi per Q3 segue un criterio preciso: accoppiare ciascuna API con l'algoritmo più naturale e rappresentativo per quella modalità di programmazione.

- **DataFrame + `percentile_approx` (Greenwald-Khanna).** `F.percentile_approx` è la funzione nativa

di Spark per il calcolo approssimato dei percentili su DataFrame. Internamente adotta l’algoritmo Greenwald-Khanna, che costruisce un summary distribuito direttamente sugli executor senza mai trasferire dati al driver. È l’approccio idiomatico per grandi dataset con l’API DataFrame: una singola chiamata dichiarativa, interamente ottimizzata dal Catalyst optimizer.

- **RDD + t-digest.** T-digest è un algoritmo alternativo basato su centroidi a densità variabile, più accurato alle code della distribuzione, dove percentili estremi come P90 sono più difficili da stimare, e naturalmente componibile: ogni partizione costruisce il proprio digest locale, i digest vengono poi fusi tramite `reduceByKey + update_centroids_from_list`.

I due metodi producono risultati numericamente vicini ma semanticamente diversi. `percentile_approx` implementa un quantile empirico discreto e restituisce sempre un valore osservato nel dataset, comportamento appropriato per variabili come `DEP_DELAY` espresso in minuti interi. T-digest effettua interpolazione lineare tra i centroidi dello sketch, restituendo valori decimali che non corrispondono a osservazioni reali. Dal punto di vista della precisione, la differenza massima osservata tra i due metodi è inferiore a 1 minuto su tutti i quantili analizzati, confermando che entrambi gli approcci sono adeguati per questo tipo di analisi.

D. Result Export (Redis)

Si è scelto di utilizzare il *key-value* datastore NoSQL *Redis*, in particolare nella distribuzione *Redis Stack (7.2.0-v10)*, come serving layer per la successiva visualizzazione dell’output in dashboard Grafana.

Redis Stack estende Redis con moduli aggiuntivi e strutture dati avanzate, tra cui *RedisTimeSeries* e *RedisJSON*, rendendolo più adatto alla memorizzazione e interrogazione di risultati strutturati rispetto a una semplice istanza Redis.

Le motivazioni principali di questa scelta sono legate alla flessibilità del modello key-value offerto da Redis, adatto alla memorizzazione di risultati aggregati di piccole dimensioni, come statistiche per mese, compagnia aerea, ed ora. Inoltre, *RedisTimeSeries* è stato utilizzato per rappresentare dati ordinati lungo un asse temporale, ad esempio mesi o ore del giorno (*Query 1* e *Query 3*), mentre Hash e List sono stati impiegati per rappresentare tabelle, metriche indicizzate e classifiche ordinate. Un ulteriore elemento a favore di questa scelta è stata la disponibilità del plugin gratuito *Redis Datasource* per Grafana, che permette di interrogare direttamente Redis dalle dashboard.

L’utilizzo di Redis come serving layer ha imposto una scelta attenta delle chiavi che permettesse un’adeguata visualizzazione in Grafana. La mappatura da CSV a key-value avviene nei due file di `export_output_to_redis.py` e `export_benchmark_to_redis.py`.

E. Visualizzazione (Grafana)

Per la visualizzazione è stato utilizzato Grafana 10.4.0, accompagnato dal plugin *redis-datasource*.

Sono state realizzate tre dashboard principali, una per ogni query.

Per la realizzazione dei singoli *panel*, i dati vengono reperiti dal datasource redis tramite query *ad hoc*, utilizzando comandi Redis compatibili con le strutture dati definite nella fase di export (`TS.RANGE`, `HGET`, etc.), e successivamente manipolati tramite l’applicazione di adeguate trasformazioni (`extract`, `organize`, `join`, `convert`, etc.).

La persistenza della configurazione Grafana è gestita tramite *provisioning* nelle relative cartelle del progetto. In particolare, sia il datasource Redis sia le dashboard vengono caricati automaticamente all’avvio del container Grafana, rendendo l’ambiente di visualizzazione riproducibile.

F. Pipeline orchestrazione (Apache AirFlow)

Il DAG `sabd_project1_pipeline` in Airflow coordina l’intero flusso seguendo la pipeline (*Figura 3*).

Airflow è configurato con SQLite come backend e SequentialExecutor, che esegue i task in sequenza anche quando definiti come paralleli nel DAG. Questa scelta è motivata dal contesto di esecuzione: l’intera architettura gira su WSL2, ambiente in cui le risorse di memoria e CPU disponibili sono limitate e condivise tra tutti i container Docker. Eseguire le tre query Spark contemporaneamente avrebbe generato contesa sulle risorse degli executor, rischiando timeout o fallimenti per esaurimento memoria. L’esecuzione sequenziale garantisce che ciascuna query disponga dell’intera capacità del cluster Spark durante la propria esecuzione, a fronte di un overhead temporale accettabile nel contesto batch del progetto.



Figura 3. DAG Airflow

IV. RISULTATI

Sono di seguito riportate le analisi dei risultati delle tre query, a partire dalla *Query 1*.

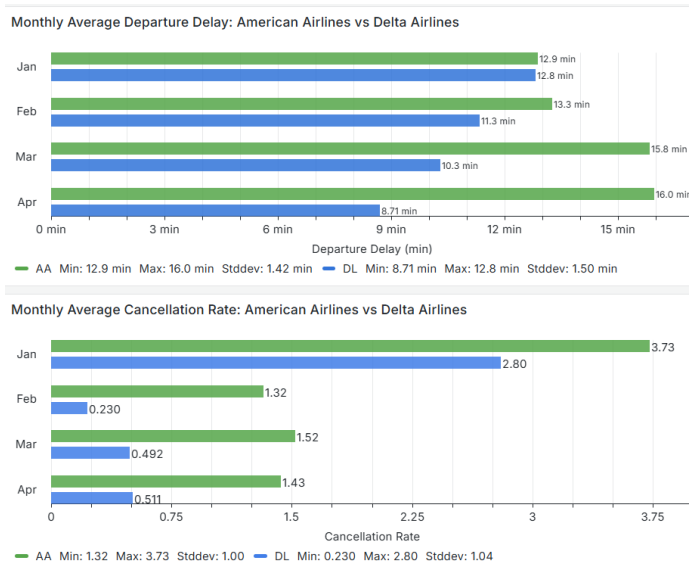


Figura 4. Risultati della Query 1

Sono riportati di seguito i risultati della Query 2: classifica delle prime dieci compagnie per ritardo medio in arrivo e contributi medi delle diverse cause di ritardo.

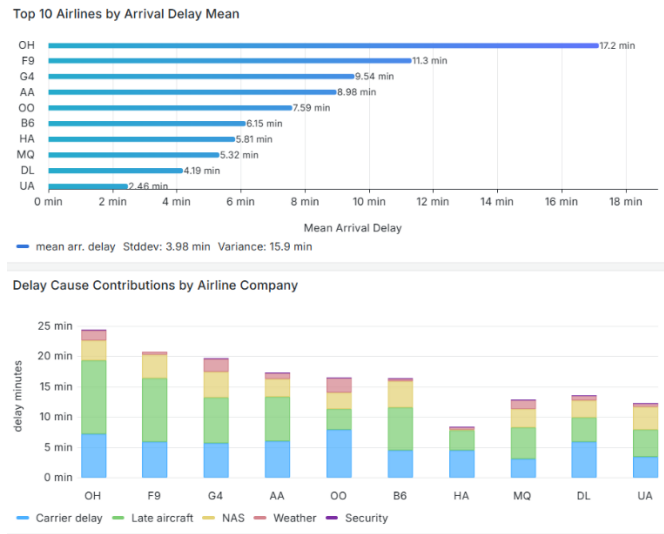


Figura 5 Risultati della Query 2

Infine, ognuno dei seguenti *panel* illustra i percentili (25th, 50th, 75th, 90th) per ognuna delle quattro compagnie richieste.

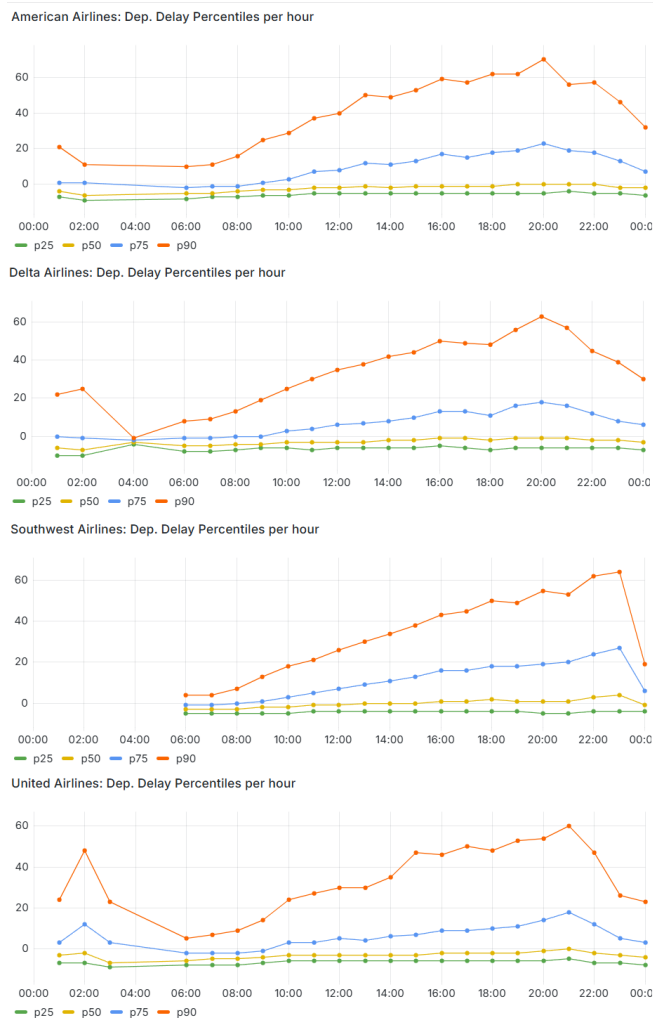


Figura 6. Risultati della Query 3

V. ANALISI DELLE PRESTAZIONI

Per analizzare le prestazioni del sistema e la qualità delle scelte progettuali, sono stati effettuati quattro esperimenti, i cui esiti hanno permesso di individuare la configurazione ottimale per la pipeline e il deploy scelti.

Il primo esperimento riguarda il confronto delle performance tra il formato Dataframe e il formato RDD. Il secondo esperimento misura le prestazioni al variare del numero di Spark Worker. Il terzo esperimento investiga l'impatto del caching della Spark Session, confrontando esecuzioni "Cold Start", dove la Spark Session viene di volta in volta ricreata, ed esecuzioni "Warm Start", dove la Spark Session è condivisa tra le iterazioni. Infine, l'ultimo esperimento valuta l'impatto del caching manuale sulle prestazioni delle query.

Tutti gli esperimenti sono stati realizzati in deploy locale sulla stessa macchina host, che possiede un processore AMD Ryzen 7 5700U Series with Radeon Graphics, 8 cores e 16GB di RAM.

A. Metriche di misurazione

Per la valutazione delle prestazioni sono state definite le seguenti metriche principali:

- **Loading time:** tempo di lettura dei dati Parquet da HDFS, inclusa l'operazione di select utilizzata per forzare il caricamento effettivo dei dati.
- **Processing time:** tempo di sola computazione della query, corrisponde alla fase `computation_s` e rappresenta il costo puro delle trasformazioni Spark.
- **End-to-end time:** tempo totale di esecuzione (dall'avvio al termine), comprensivo di setup Spark, I/O e overhead di sistema.

La raccolta dei tempi è avvenuta prestando attenzione al meccanismo di *lazy evaluation* di Spark: per misurare il tempo di una fase, si calcola la differenza tra l'istante antecedente le trasformazioni e l'istante subito successivo alla prima azione utile che ne forzi l'esecuzione.

B. DataFrame vs RDD

Il confronto prestazionale tra le implementazioni basate su API DataFrame e quelle sviluppate tramite RDD è stato condotto su tutte e tre le query analitiche, utilizzando un singolo Spark Worker. I risultati sperimentali di tipo *end-to-end*, sintetizzati nella *Tabella 1*, evidenziano una netta e strutturale discrepanza nei tempi misurati, sottolineando i vantaggi dell'astrazione di alto livello (sfrutta il Catalyst Optimizer) rispetto a quella di basso livello.

Tabella 1: Confronto DataFrame vs RDD (end-to-end time)

Query	API	Mean	Std. Dev.
Q1	DataFrame	2.91	0.53
Q1	RDD	40.09	1.78
Q2	DataFrame	3.38	0.39
Q2	RDD	62.70	0.76
Q3	DataFrame	18.78	1.19
Q3	RDD	24.27	0.31

Tabella 2: Confronto DataFrame vs RDD (processing time)

Query	API	Mean	Std. Dev.
Q1	DataFrame	0.68	0.13
Q1	RDD	38.95	1.82

Q2	DataFrame	1.06	0.10
Q2	RDD	61.26	0.81
Q3	DataFrame	4.32	0.75
Q3	RDD	21.95	0.29

C. Scalabilità del numero di worker

Ai fini dell'analisi di scalabilità, il benchmark è stato eseguito con numero variabile di worker, da un minimo di 1 a un massimo di 4. Per permettere l'esecuzione locale, gli Spark Worker sono stati limitati a 1GB di memoria e 1 core (`docker_compose.yml`). Per ogni configurazione (query; formato; numero di worker), sono state eseguite 20 iterazioni, di cui 5 di warm-up escluse dalle statistiche e 15 considerate valide.

Dall'analisi dei risultati (Tabelle 3 e 4, Figure 6, 7, e 8, 9) si evince che, per la configurazione scelta, l'utilizzo di 2 worker rappresenta il miglior compromesso tra tempo di esecuzione e risorse impiegate. Tale conclusione deriva dall'andamento delle curve di scaling: il passaggio da 1 a 2 worker produce una riduzione significativa dei tempi, mentre l'aggiunta di ulteriori worker non comporta miglioramenti proporzionati e, in alcuni casi, introduce un significativo peggioramento delle performance (Figura 8, API RDD).

Tabella 3: Risultati dell'analisi di scaling – End-to-End Time

Query	API	1W	2W	3W	4W
Q1	DataFrame	2.91 ± 0.53	1.73 ± 0.21	1.70 ± 0.32	2.07 ± 0.28
	RDD	40.09 ± 1.78	9.49 ± 0.68	14.32 ± 1.20	61.88 ± 157.30
Q2	DataFrame	3.38 ± 0.39	1.42 ± 0.17	1.53 ± 0.16	1.63 ± 0.29
	RDD	62.70 ± 0.76	12.92 ± 0.56	12.71 ± 0.62	12.63 ± 0.67
Q3	DataFrame	18.78 ± 1.19	4.58 ± 0.24	4.75 ± 0.53	5.27 ± 0.70
	RDD	24.27 ± 0.31	15.74 ± 1.77	15.23 ± 1.07	15.59 ± 0.77

I Valori sono riportati come media ± deviazione standard, calcolate sul totale delle 15 iterazioni valide. 1W = 1 Worker.

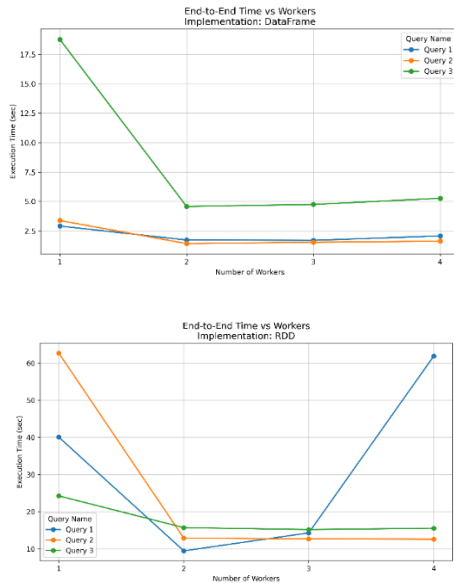


Figura 7. Confronto dei tempi End-to-end (Dataframe vs RDD)

Tabella 4: Risultati dell'analisi di scaling – Processing Time

Query	API	1W	2W	3W	4W
Q1	DataFrame	0.68 ± 0.13	0.46 ± 0.04	0.46 ± 0.09	0.57 ± 0.09
	RDD	38.95 ± 1.82	8.83 ± 0.34	13.71 ± 1.12	60.99 ± 157.13
Q2	DataFrame	1.06 ± 0.10	0.48 ± 0.04	0.56 ± 0.05	0.51 ± 0.04
	RDD	61.26 ± 0.81	12.04 ± 0.39	11.66 ± 0.61	11.32 ± 0.61
Q3	DataFrame	4.32 ± 0.75	0.86 ± 0.06	0.91 ± 0.14	1.06 ± 0.19
	RDD	21.95 ± 0.29	13.57 ± 1.01	13.23 ± 1.04	13.17 ± 0.75

I Valori sono riportati come media ± deviazione standard, calcolate sul totale delle 15 iterazioni valide. 1W = 1 Worker.

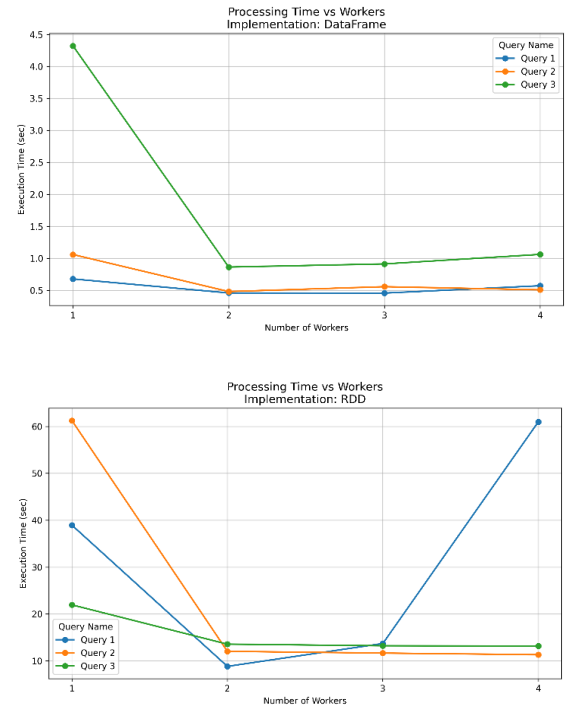


Figura 8. Confronto dei tempi di processing (Dataframe vs RDD)

D. Warm start vs Cold start

Per valutare l'impatto del caching della Sessione Spark, sono stati realizzate due diversi benchmark:

- **Benchmark warm start:** 20 iterazioni totali per query e per implementazione, di cui le prime 5 di warm-up escluse dalla statistica. Le 15 iterazioni valide misurano separatamente le quattro fasi (`loading_s`, `computation_s`, `output_s`, `end_to_end_s`). La sessione Spark è condivisa tra le iterazioni: a partire dalle prime esecuzioni, il Parquet risulta già in cache a livello OS/JVM, simulando un carico operativo realistico. La scrittura CSV viene eseguita ad ogni iterazione valida per garantire a `output_s` la stessa consistenza statistica delle altre fasi.
- **Benchmark cold start:** 15 iterazioni totali per query, senza iterazioni di warm-up. La sessione Spark viene

terminata e ricreata tra un'iterazione e l'altra, azzerando ogni forma di caching in memoria. Questo scenario simula l'esecuzione della pipeline in condizioni di avvio a freddo, misurando il costo reale del primo accesso ai dati.

I risultati in Tabella 5 sono relativi all'esecuzione del benchmark con 1 Spark Worker e API DataFrame.

Tabella 5: Confronto risultati benchmark Cold Start vs Warm Start

Query	Phase	Mode	Mean (s)	StdDev
Q1	Spark start	cold start	0.70	1.33
Q1	Loading	cold start	14.60	1.75
Q1	Loading	warmup	0.55	0.04
Q1	Processing	cold start	6.16	0.95
Q1	Processing	warmup	0.77	0.07
Q1	End-to-end	cold start	25.37	4.43
Q1	End-to-end	warmup	2.86	0.45
Q1	Spark stop	cold start	0.39	0.14
Q2	Spark start	cold start	0.33	0.03
Q2	Loading	cold start	13.94	0.36
Q2	Loading	warmup	0.45	0.03
Q2	Processing	cold start	7.73	0.31
Q2	Processing	warmup	0.79	0.07
Q2	End-to-end	cold start	26.36	0.59
Q2	End-to-end	warmup	2.83	0.20
Q2	Spark stop	cold start	0.43	0.12
Q3	Spark start	cold start	0.32	0.02
Q3	Loading	cold start	14.20	0.43
Q3	Loading	warmup	0.39	0.04
Q3	Processing	cold start	15.20	0.55
Q3	Processing	warmup	8.05	0.19
Q3	End-to-end	cold start	41.41	1.19
Q3	End-to-end	warmup	15.14	0.36
Q3	Spark stop	cold start	0.30	0.09

I risultati mostrano un impatto significativo del warm-up su tutte le fasi di esecuzione. La differenza più marcata riguarda il loading: in cold start la lettura del Parquet richiede circa 3.5s per tutte le query, mentre in warm session scende a 0.06-0.09s, confermando che dopo le prime iterazioni il file viene mantenuto in cache OS/JVM. Anche il processing migliora sensibilmente grazie alla JIT compilation della JVM, che ottimizza il bytecode a runtime dopo aver identificato i percorsi di codice più caldi (*hotspots*).

La variabilità delle misurazioni è notevolmente più alta in cold start, riflettendo l'instabilità del primo accesso a disco e del riscaldamento della JVM. In warm session la variabilità si riduce drasticamente, confermando la stabilità delle misurazioni nelle iterazioni valide.

E. Impatto del caching

Un'ultima osservazione è dovuta all'impatto dell'utilizzo di `.cache()` da parte dello sviluppatore all'interno delle query, in quanto ne influenza sia la misurazione dei tempi sia il comportamento reale. Applicare la cache a ogni DataFrame intermedio permette di separare in modo più netto *loading*, *computation* e *output*, ma introduce materializzazioni aggiuntive, aumenta la pressione sulla memoria e può ridurre la libertà di ottimizzazione del piano da parte di Spark (*Catalyst Optimizer*). Nei test questo si traduce in tempi più alti nelle fasi iniziali, che includono anche il costo di

popolamento della cache, e in tempi di *processing* più bassi perché basati su dati già materializzati.

Per evitare di misurare una versione artificiale della query, la cache è stata quindi mantenuta solo dove giustificata da un riutilizzo effettivo del DataFrame, ad esempio quando un risultato viene usato da più action o da più output. La metodologia finale privilegia quindi tempi più realistici, accettando una separazione delle fasi meno perfetta ma più rappresentativa dell'esecuzione Spark reale.

Tabella 6: Confronto risultati – impatto del caching – Warm Session

Query	Phase	Impl.	Mean (s)	StdDev
Q1	Loading	cache	2.36	0.23
Q1	Loading	no cache	0.55	0.04
Q1	Processing	cache	0.43	0.06
Q1	Processing	no cache	0.77	0.07
Q1	Output	cache	0.65	0.25
Q1	Output	no cache	0.89	0.36
Q1	End-to-end	cache	4.37	0.53
Q1	End-to-end	no cache	2.86	0.45

*Cache = caching effettuato in ogni fase intermedia.
No cache = cache effettuato solo dove necessario (baseline)*

VI. CONCLUSIONI

I risultati sperimentali confermano la superiorità dell'API DataFrame rispetto all'API RDD per tutte e tre le query analizzate. Tale vantaggio è riconducibile principalmente all'ottimizzatore Catalyst, che pianifica ed esegue automaticamente il piano di query in modo efficiente, a differenza dell'API RDD che non beneficia di tali ottimizzazioni e introduce overhead aggiuntivi legati alla serializzazione e alla gestione esplicita delle trasformazioni.

Relativamente allo scaling, la configurazione con un numero ridotto di worker si è rivelata ottimale nell'ambiente considerato. Questo risultato è strettamente dipendente dal contesto di deployment (nodo standalone su Docker Compose e WSL2 con risorse limitate e condivise) dove l'overhead di coordinazione inter-container tende a ridurre i guadagni derivanti da una maggiore parallelizzazione. Pertanto, i risultati sullo scaling potrebbero differire significativamente su infrastruttura fisica multi-nodo.

L'effetto del caching di Spark sulle prestazioni è risultato marcato, in particolare sulla metrica *computation_s*, coerentemente con il design fondamentale del framework, che riduce il costo dei ricalcoli mantenendo i dataset intermedi in memoria RAM.

Nel complesso, il progetto mostra come una pipeline batch basata su NiFi, HDFS, Spark, Redis e Grafana possa essere realizzata in modo end-to-end in un ambiente containerizzato, coprendo ingestion, preprocessing, interrogazioni analitiche, benchmark, serving dei risultati e visualizzazione. Dal punto di vista prestazionale, l'esperimento evidenzia il vantaggio dell'utilizzo delle API dichiarative di Spark rispetto a implementazioni più manuali basate su RDD, ma anche l'importanza del contesto infrastrutturale nella valutazione dello scaling. Sviluppi futuri potrebbero includere l'esecuzione dei benchmark su un cluster distribuito reale e ulteriori analisi prestazionali.

End-to-end execution time distribution

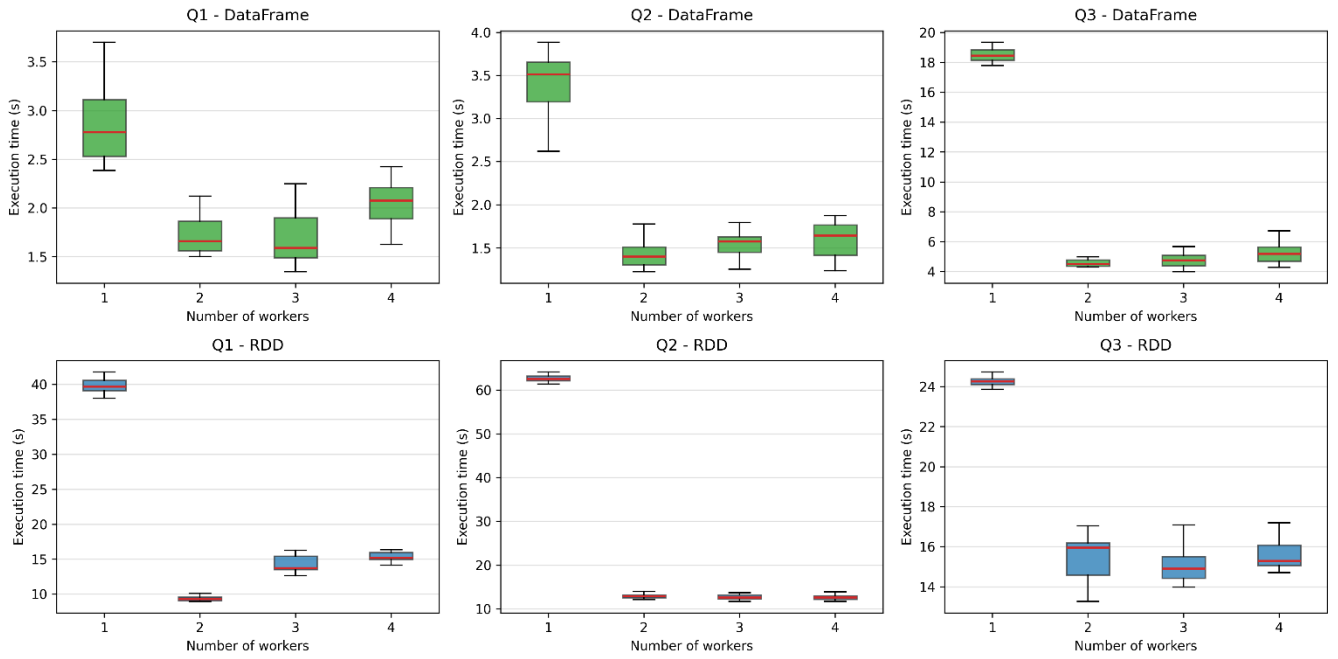


Figura 9. End to End time per query. Nota: gli outliers non vengono mostrati per leggibilità.

Processing time distribution

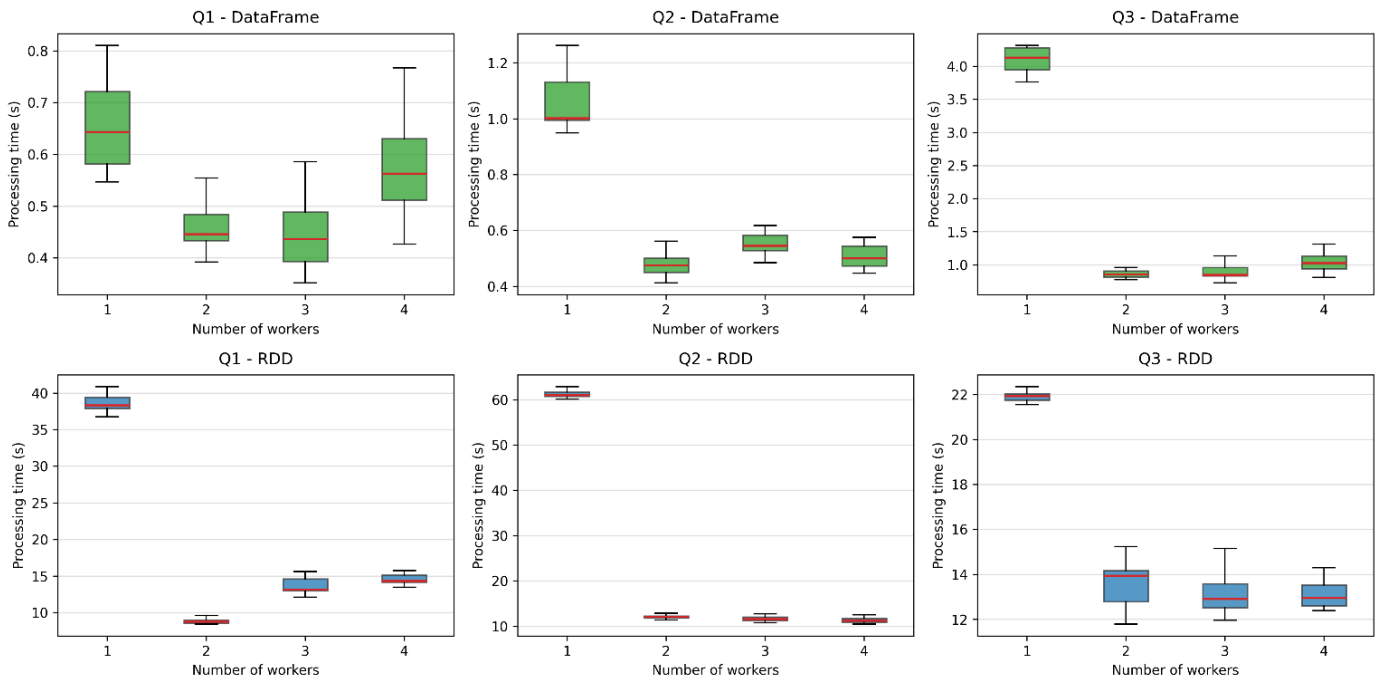


Figura 10. Processing time per query. Nota: gli outliers non vengono mostrati per leggibilità.

REFERENCES

- [1] Apache Software Foundation. Apache Spark. Available: <https://spark.apache.org/>
- [2] U.S. Department of Transportation. Bureau of Transportation Statistics (BTS). Available: <https://www.transtats.bts.gov/>
- [3] Docker. Available: <https://www.docker.com/>
- [4] Apache Software Foundation. Apache Airflow. Available: <https://airflow.apache.org/>
- [5] Apache Software Foundation. Apache NiFi. Available: <https://nifi.apache.org/>
- [6] Apache Software Foundation. Hadoop Distributed File System (HDFS). Available: <https://hadoop.apache.org/>
- [7] Redis. Available: <https://redis.io/>
- [8] Grafana Labs. Grafana: The open observability platform. Available: <https://grafana.com/grafana/>
- [9] Redis Stack. <https://redis.io/about/redis-stack/>
- [10] T. Dunning. The t-digest: Efficient estimates of distributions. *Software Impacts*, 7:100049, 2021. <https://www.sciencedirect.com/science/article/pii/S2665963820300403>